

DATABASE PROJECT REPORT

ON-DEMAND STREAMING PLATFORM (MINI-NETFLIX)

Due Date: [29/08 - Week 4]

Project ID & Title: On-Demand Streaming Platform (Mini-Netflix)

Team Name: [ba chang linh ngu lam]

Team Members:

1. [Lê Ngọc Dũng (n24dece065@student.ptithcm.edu.vn)]
2. [Lê Phạm Khánh Huy(n24dece072@student.ptithcm.edu.vn)]
3. [Phan Giảng Bình(n24dece056@student.ptithcm.edu.vn)]

Project Title:

Database Design and Implementation for an On-Demand Streaming Platform

TABLE OF CONTENTS

A. Project Identity

1. Introduction & Project Scope

1.1 System Objective

1.2 Business Rules & Constraints

2. Database Design

2.1 Conceptual Model (ER/EER Diagram)

2.2 Logical Schema Mapping

2.3 Normalization Verification (1NF $\rightarrow$ 3NF/BCNF)

3. Data Dictionary

4. Database Implementation

4.1 DDL Script (Tables, Views, Indexes, Triggers)

4.2 Advanced Queries & Performance Test Cases

5. Verification & Security

5.1 Test Cases

5.2 Role-Based Access Control (RBAC)

A. PROJECT IDENTITY

Item	Information
Project ID & Title	[Insert Project ID]: On-Demand Streaming Platform (Mini-Netflix)
Team Name	[Insert Team Name]
Team Members	[Insert names and student emails]
Project Title	Database Design and Implementation for an On-Demand Streaming Platform
Project Domain	Online video streaming / digital entertainment
Database Focus	EER hierarchy, M:N mapping, normalization, constraints, SQL security

1. INTRODUCTION & PROJECT SCOPE

1.1 System Objective

The objective of this project is to design and implement a relational database for an on-demand streaming platform similar to a simplified Netflix system. The database manages user accounts, multiple profiles, content categorization, movies, TV series, episodes, viewing history, subscription plans, and user subscriptions.

The project applies an Enhanced Entity Relationship (EER) model to represent inheritance in the content domain. CONTENT is modeled as a supertype, while MOVIES and EPISODE are subtypes. The many-to-many relationship between PROFILE and CONTENT is implemented through WATCH_HISTORY, which stores viewing progress and completion status.

1.2 Business Rules & Constraints

- Each USER is uniquely identified by USER_ID and should have a unique USERNAME and EMAIL.
- One USER can own multiple PROFILE records; each PROFILE belongs to exactly one USER.
- The number of profiles should not exceed MAX_PROFILES of the user's active subscription plan; this can be enforced by a trigger or application logic.
- Each CONTENT record is uniquely identified by CONTENT_ID.
- A CONTENT item is specialized as either a MOVIE or an EPISODE in the current design.
- Each MOVIE and EPISODE must correspond to a CONTENT record.
- A TV_SERIES can contain many EPISODE records; an EPISODE belongs to one TV_SERIES.
- A PROFILE can watch many CONTENT items and a CONTENT item can be watched by many PROFILE records.
- WATCH_DURATION must be non-negative and must not exceed the duration of the content.
- Each SUBSCRIPTION references one USER and one SUBSCRIPTION_PLAN.
- END_DATE must be greater than or equal to START_DATE.
- STATUS is restricted to ACTIVE, EXPIRED, or CANCELLED.

2. DATABASE DESIGN (ISO/IEC 19505 / IE STANDARDS)

2.1 Conceptual Model (ER/EER Diagram)

The conceptual model contains nine main entities: USERS, PROFILE, CONTENT, MOVIES, TV_SERIES, EPISODE, WATCH_HISTORY, SUBSCRIPTION_PLAN, and SUBSCRIPTION. The central EER hierarchy is CONTENT → {MOVIES, EPISODE}.

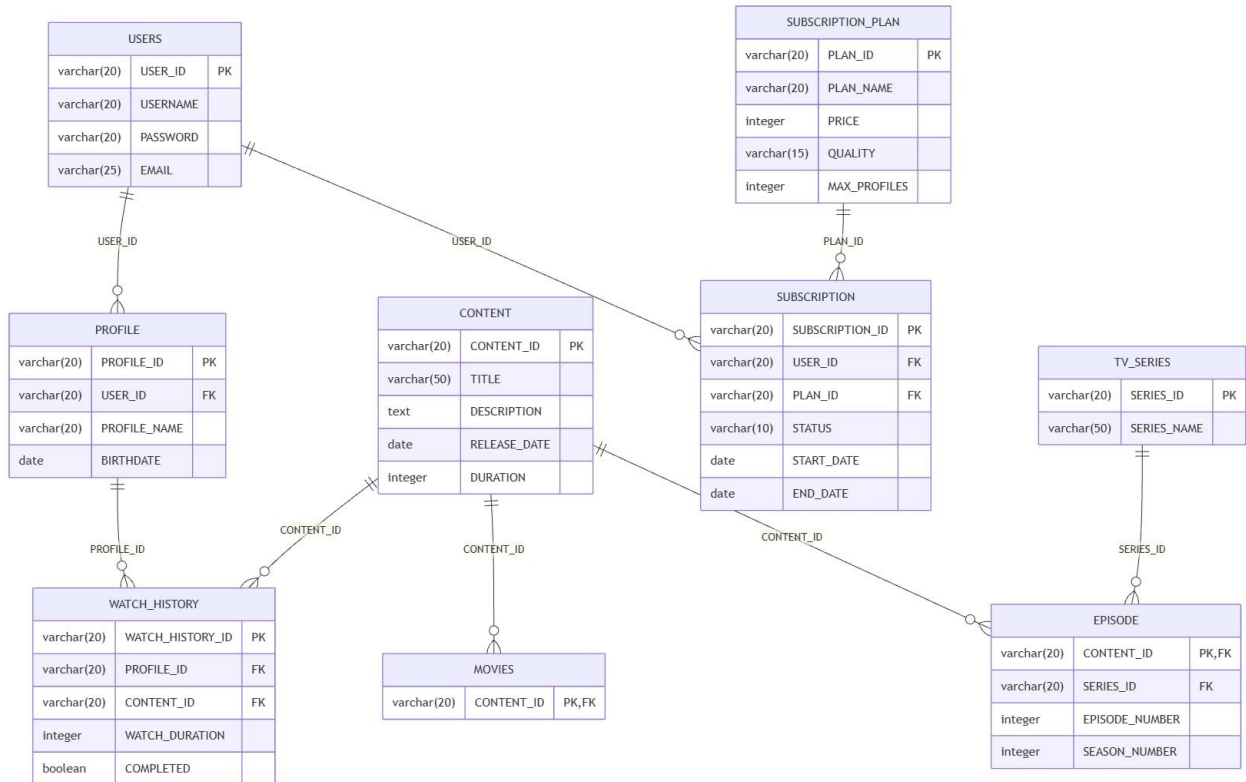


Figure 1. EER Diagram of the Mini-Netflix Database

2.2 Logical Schema Mapping

Relation	Primary Key	Foreign Keys
USERS(user_id, username, password hash, email)	user_id	—
PROFILE(profile_id, user_id, profile name, birthdate)	profile_id	user_id → USERS
CONTENT(content_id, title, description, release date, duration)	content_id	—
MOVIES(content_id)	content_id	content_id → CONTENT
TV_SERIES(series_id, series_name)	series_id	—
EPISODE(content_id, series_id, episode number, season number)	content_id	content_id → CONTENT; series_id → TV_SERIES
WATCH_HISTORY(watch_history_id, profile_id, content_id, watch_duration, completed)	watch_history_id	profile_id → PROFILE; content_id → CONTENT
SUBSCRIPTION PLAN(plan_id, plan_name, price, quality, max_profiles)	plan_id	—

plan_name, price, quality, max_profiles)		
SUBSCRIPTION(subscription_id, user_id, plan_id, status, start_date, end_date)	subscription_id	user_id → USERS; plan_id → SUBSCRIPTION_PLAN

2.3 Normalization Verification (Formal Proofs for 1NF → 3NF/BCNF)

2.3.1 First Normal Form (1NF)

All relations contain atomic values and no repeating groups. Episodes are stored as rows instead of repeated episode columns. Therefore, all relations satisfy 1NF.

2.3.2 Second Normal Form (2NF)

Every relation is in 1NF and all non-key attributes depend on the whole candidate key. Most relations have a single-attribute primary key, so partial dependency does not occur. Therefore, the schema satisfies 2NF.

2.3.3 Third Normal Form (3NF)

Transitive dependencies are removed by separating independent entities. For example, PLAN_NAME, PRICE, QUALITY, and MAX_PROFILES are stored in SUBSCRIPTION_PLAN rather than SUBSCRIPTION. SERIES_NAME is stored in TV_SERIES rather than EPISODE.

USERS:

user_id → username, password_hash, email

PROFILE:

profile_id → user_id, profile_name, birthdate

CONTENT:

content_id → title, description, release_date, duration

TV_SERIES:

series_id → series_name

SUBSCRIPTION_PLAN:

plan_id → plan_name, price, quality, max_profiles

SUBSCRIPTION:

subscription_id → user_id, plan_id, status, start_date, end_date

Each determinant above is a key for its relation and descriptive attributes are stored with the entity they describe. Thus the main schema satisfies 3NF.

2.3.4 BCNF

Under the stated functional dependencies, the determinant of every non-trivial dependency is a candidate key. Therefore, the main relations are compatible with BCNF. Unique constraints on

USERNAME, EMAIL, PLAN_NAME, episode position, and profile-content history further improve integrity.

3. DATA DICTIONARY (ADAPTED FROM ISO/IEC 11179)

Table	Attribute	Data Type	Constraints	Definition
USERS	USER_ID	VARCHAR(20)	PK	Unique user identifier
USERS	USERNAME	VARCHAR(20)	UNIQUE, NOT NULL	Login name
USERS	PASSWORD_HASH	VARCHAR(255)	NOT NULL	Password hash
USERS	EMAIL	VARCHAR(100)	UNIQUE, NOT NULL	User email
PROFILE	PROFILE_ID	VARCHAR(20)	PK	Profile identifier
PROFILE	USER_ID	VARCHAR(20)	FK	Owner user
PROFILE	PROFILE_NAME	VARCHAR(20)	NOT NULL	Display name
PROFILE	BIRTHDATE	DATE	NULL	Birth date
CONTENT	CONTENT_ID	VARCHAR(20)	PK	Content identifier
CONTENT	TITLE	VARCHAR(50)	NOT NULL	Content title
CONTENT	DESCRIPTION	TEXT	NULL	Content description
CONTENT	RELEASE_DATE	DATE	NULL	Release date
CONTENT	DURATION	INTEGER	CHECK >= 0	Duration in minutes
MOVIES	CONTENT_ID	VARCHAR(20)	PK, FK	Movie identifier
TV_SERIES	SERIES_ID	VARCHAR(20)	PK	Series identifier
TV_SERIES	SERIES_NAME	VARCHAR(50)	NOT NULL	Series name
EPISODE	CONTENT_ID	VARCHAR(20)	PK, FK	Episode content
EPISODE	SERIES_ID	VARCHAR(20)	FK	Parent series
EPISODE	EPISODE_NUMBER	INTEGER	CHECK > 0	Episode sequence
EPISODE	SEASON_NUMBER	INTEGER	CHECK > 0	Season sequence
WATCH_HISTORY	WATCH_HISTORY_I	VARCHAR(20)	PK	History

	D			identifier
WATCH_HISTORY	PROFILE_ID	VARCHAR(20)	FK	Viewer profile
WATCH_HISTORY	CONTENT_ID	VARCHAR(20)	FK	Viewed content
WATCH_HISTORY	WATCH_DURATION	INTEGER	CHECK >= 0	Minutes watched
WATCH_HISTORY	COMPLETED	BOOLEAN	NOT NULL	Completion status
SUBSCRIPTION_PLAN	PLAN_ID	VARCHAR(20)	PK	Plan identifier
SUBSCRIPTION_PLAN	PLAN_NAME	VARCHAR(20)	UNIQUE	Plan name
SUBSCRIPTION_PLAN	PRICE	INTEGER	CHECK >= 0	Plan price
SUBSCRIPTION_PLAN	QUALITY	VARCHAR(15)	NOT NULL	Streaming quality
SUBSCRIPTION_PLAN	MAX_PROFILES	INTEGER	CHECK > 0	Maximum profiles
SUBSCRIPTION	SUBSCRIPTION_ID	VARCHAR(20)	PK	Subscription identifier
SUBSCRIPTION	USER_ID	VARCHAR(20)	FK	Subscribed user
SUBSCRIPTION	PLAN_ID	VARCHAR(20)	FK	Selected plan
SUBSCRIPTION	STATUS	VARCHAR(10)	CHECK	Subscription state
SUBSCRIPTION	START_DATE	DATE	NOT NULL	Start date
SUBSCRIPTION	END_DATE	DATE	CHECK	End date

4. DATABASE IMPLEMENTATION (SQL STYLE GUIDE COMPLIANT)

The following implementation uses PostgreSQL-style SQL. SQL keywords are uppercase and identifiers use snake_case.

4.1 DDL Script (Tables, Views, Indexes, Triggers)

4.1.1 Tables

```
CREATE TABLE users (
    user_id VARCHAR(20) PRIMARY KEY,
    username VARCHAR(20) NOT NULL UNIQUE,
    password_hash VARCHAR(255) NOT NULL,
    email VARCHAR(100) NOT NULL UNIQUE
);
```

```
CREATE TABLE subscription_plan (
    plan_id VARCHAR(20) PRIMARY KEY,
```

```

    plan_name VARCHAR(20) NOT NULL UNIQUE,
    price INTEGER NOT NULL CHECK (price >= 0),
    quality VARCHAR(15) NOT NULL,
    max_profiles INTEGER NOT NULL CHECK (max_profiles > 0)
);

CREATE TABLE profile (
    profile_id VARCHAR(20) PRIMARY KEY,
    user_id VARCHAR(20) NOT NULL,
    profile_name VARCHAR(20) NOT NULL,
    birthdate DATE,
    FOREIGN KEY (user_id) REFERENCES users(user_id) ON DELETE CASCADE
);

CREATE TABLE content (
    content_id VARCHAR(20) PRIMARY KEY,
    title VARCHAR(50) NOT NULL,
    description TEXT,
    release_date DATE,
    duration INTEGER NOT NULL CHECK (duration >= 0)
);

CREATE TABLE movies (
    content_id VARCHAR(20) PRIMARY KEY,
    FOREIGN KEY (content_id) REFERENCES content(content_id) ON DELETE CASCADE
);

CREATE TABLE tv_series (
    series_id VARCHAR(20) PRIMARY KEY,
    series_name VARCHAR(50) NOT NULL
);

CREATE TABLE episode (
    content_id VARCHAR(20) PRIMARY KEY,
    series_id VARCHAR(20) NOT NULL,
    episode_number INTEGER NOT NULL CHECK (episode_number > 0),
    season_number INTEGER NOT NULL CHECK (season_number > 0),
    FOREIGN KEY (content_id) REFERENCES content(content_id) ON DELETE CASCADE,
    FOREIGN KEY (series_id) REFERENCES tv_series(series_id) ON DELETE CASCADE,
    UNIQUE (series_id, season_number, episode_number)
);

CREATE TABLE watch_history (
    watch_history_id VARCHAR(20) PRIMARY KEY,
    profile_id VARCHAR(20) NOT NULL,
    content_id VARCHAR(20) NOT NULL,
    watch_duration INTEGER NOT NULL CHECK (watch_duration >= 0),
    completed BOOLEAN NOT NULL DEFAULT FALSE,
    FOREIGN KEY (profile_id) REFERENCES profile(profile_id) ON DELETE CASCADE,
    FOREIGN KEY (content_id) REFERENCES content(content_id) ON DELETE CASCADE,
    UNIQUE (profile_id, content_id)
);

```

```

CREATE TABLE subscription (
    subscription_id VARCHAR(20) PRIMARY KEY,
    user_id VARCHAR(20) NOT NULL,
    plan_id VARCHAR(20) NOT NULL,
    status VARCHAR(10) NOT NULL
        CHECK (status IN ('ACTIVE', 'EXPIRED', 'CANCELLED')),
    start_date DATE NOT NULL,
    end_date DATE NOT NULL CHECK (end_date >= start_date),
    FOREIGN KEY (user_id) REFERENCES users(user_id),
    FOREIGN KEY (plan_id) REFERENCES subscription_plan(plan_id)
);

```

4.1.2 View

```

CREATE VIEW v_profile_watch_history AS
SELECT p.profile_id, p.profile_name, c.content_id, c.title,
       w.watch_duration, w.completed
FROM watch_history w
JOIN profile p ON p.profile_id = w.profile_id
JOIN content c ON c.content_id = w.content_id;

```

4.1.3 Indexes

```

CREATE INDEX idx_profile_user ON profile(user_id);
CREATE INDEX idx_episode_series ON episode(series_id, season_number, episode_number);
CREATE INDEX idx_watch_history_profile ON watch_history(profile_id);
CREATE INDEX idx_watch_history_content ON watch_history(content_id);
CREATE INDEX idx_subscription_user_status ON subscription(user_id, status);

```

4.1.4 Trigger

```

CREATE OR REPLACE FUNCTION validate_watch_duration()
RETURNS TRIGGER AS $$
DECLARE max_duration INTEGER;
BEGIN
    SELECT duration INTO max_duration
    FROM content
    WHERE content_id = NEW.content_id;

    IF NEW.watch_duration > max_duration THEN
        RAISE EXCEPTION 'Watch duration cannot exceed content duration';
    END IF;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trg_validate_watch_duration
BEFORE INSERT OR UPDATE ON watch_history
FOR EACH ROW EXECUTE FUNCTION validate_watch_duration();

```

4.2 Advanced Queries & Performance Test Cases

4.2.1 Viewing History of a Profile

```

SELECT p.profile_name, c.title, w.watch_duration, w.completed
FROM watch_history w
JOIN profile p ON p.profile_id = w.profile_id
JOIN content c ON c.content_id = w.content_id

```

```
WHERE p.profile_id = 'P001';
```

4.2.2 Episodes of a TV Series

```
SELECT ts.series_name, e.season_number, e.episode_number, c.title
FROM episode e
JOIN tv_series ts ON ts.series_id = e.series_id
JOIN content c ON c.content_id = e.content_id
WHERE e.series_id = 'S001'
ORDER BY e.season_number, e.episode_number;
```

4.2.3 Active Subscription for Each User

```
SELECT u.username, sp.plan_name, sp.quality, s.start_date, s.end_date
FROM subscription s
JOIN users u ON u.user_id = s.user_id
JOIN subscription_plan sp ON sp.plan_id = s.plan_id
WHERE s.status = 'ACTIVE';
```

4.2.4 Most Watched Content

```
SELECT c.content_id, c.title, COUNT(*) AS viewer_count,
       SUM(w.watch_duration) AS total_watch_duration
FROM watch_history w
JOIN content c ON c.content_id = w.content_id
GROUP BY c.content_id, c.title
ORDER BY viewer_count DESC, total_watch_duration DESC
LIMIT 10;
```

4.2.5 Performance Test Cases

Compare query execution before and after indexes using EXPLAIN ANALYZE. Actual measurements should be captured from the team's database environment.

```
EXPLAIN ANALYZE
SELECT * FROM watch_history WHERE profile_id = 'P001';
```

```
EXPLAIN ANALYZE
SELECT * FROM episode
WHERE series_id = 'S001'
ORDER BY season_number, episode_number;
```

5. VERIFICATION & SECURITY

5.1 Test Cases Showing Constraints Prevent Bad Data Insertion

Test ID	Invalid Input	Expected Result	Constraint
TC01	Duplicate USERNAME	Rejected	UNIQUE(username)
TC02	Duplicate EMAIL	Rejected	UNIQUE(email)
TC03	Unknown USER_ID in PROFILE	Rejected	Foreign key
TC04	EPISODE_NUMBER = 0	Rejected	CHECK > 0
TC05	Negative PRICE	Rejected	CHECK price >= 0

TC06	END_DATE before START_DATE	Rejected	CHECK dates
TC07	Invalid STATUS	Rejected	CHECK status
TC08	Negative WATCH_DURATION	Rejected	CHECK >= 0
TC09	WATCH_DURATION > CONTENT duration	Rejected	Trigger
TC10	Duplicate episode position	Rejected	UNIQUE(series, season, episode)

5.2 Role-Based Access Control (RBAC) Definition

The RBAC model separates administration, content management, and ordinary application usage.

```

CREATE ROLE db_admin;
CREATE ROLE content_manager;
CREATE ROLE application_user;

GRANT ALL PRIVILEGES
ON ALL TABLES IN SCHEMA public
TO db_admin;

GRANT SELECT, INSERT, UPDATE
ON content, movies, tv_series, episode
TO content_manager;

GRANT SELECT
ON subscription_plan
TO content_manager;

GRANT SELECT
ON content, movies, tv_series, episode, subscription_plan
TO application_user;

GRANT SELECT, INSERT, UPDATE
ON profile, watch_history
TO application_user;

REVOKE SELECT (password_hash)
ON users
FROM application_user;

```

In a production environment, password hashes must never be exposed to ordinary users. Direct table access should preferably be replaced by secure APIs, views, or stored procedures.

CONCLUSION

This report presents a complete database project design for an On-Demand Streaming Platform (Mini-Netflix). The main EER feature is the CONTENT supertype with MOVIES and EPISODE as

subtypes. The many-to-many relationship between PROFILE and CONTENT is implemented through WATCH_HISTORY.

The schema is normalized through 3NF and is compatible with BCNF under the stated functional dependencies. The implementation includes DDL, constraints, indexes, a view, a trigger, advanced queries, negative test cases, and RBAC definitions. The design therefore follows the required report structure and provides a strong foundation for implementation and future extension.